

Color Coding 乱択アルゴリズム グラフ理論 確率

Color-Coding

1. 概要

本講座では、**Color-Coding** という手法について解説します。

Color-Coding とは k 個のものを選ぶというような存在判定問題や最適化問題において、それぞれのものに**ランダムな色を割り当てる**という手法です。特に、比較的小さな k について「ちょうど k 種類のを 1 度ずつ使う必要がある」という制約がある最適化問題で強力です。学術的にはグラフから k 頂点のパスやサイクル、その他の種類の部分グラフを検出するアルゴリズムとして提案された乱択アルゴリズムです（[文献 1](#)）。

本講座ではそのうちでも最も簡潔な、パスの検出という問題を通して Color-Coding について解説します。

2. 前提知識

グラフの用語および、初歩的な dp を仮定します。

3. Color-Coding によるパスの検出

3.1. 問題

【問題 1】

G を N 頂点 M 辺の単純無向グラフとします。 k を正整数とします。 G に k 頂点のパス（長さ $k - 1$ のパス）が存在するか否かを判定してください。

ほとんどパスの定義そのものですが、説明のため、次の形に言い換えておきます。

【問題 1 の言い換え】

G を N 頂点 M 辺の単純無向グラフとします。 k を正整数とします。 頂点列 $(v_1, v_2, \dots, v_k)$ であって、以下の条件を満たすものが存在するか否かを判定してください。

- G のウォークである。つまり各 i ($1 \leq i \leq k-1$) に対して v_i, v_{i+1} は G において隣接する。
- $i \neq j$ ならば $v_i \neq v_j$ が成り立つ。

3.2. Color-Coding による解法

Color-Coding は頂点にランダムに色を割り当てることで、 【問題 1】 を、 次の 【問題 2】 や 【問題 3】 を繰り返し解く問題へと変換する手法です。

【問題 2】

G を N 頂点 M 辺の単純無向グラフとします。 k を正整数とします。 また各頂点には 1 以上 k 以下の整数の色が割り当てられています。

頂点列 $(v_1, v_2, \dots, v_k)$ であって、以下の条件を満たすものが存在するか否かを判定してください。

- G のウォークである。つまり各 i ($1 \leq i \leq k-1$) に対して v_i, v_{i+1} は G において隣接する。
- v_i の色は i である。

【問題 3】

G を N 頂点 M 辺の単純無向グラフとします。 k を正整数とします。 また各頂点には 1 以上 k 以下の整数の色が割り当てられています。

頂点列 $(v_1, v_2, \dots, v_k)$ であって、以下の条件を満たすものが存在するか否かを判定してください。

- G のウォークである。つまり各 i ($1 \leq i \leq k-1$) に対して v_i, v_{i+1} は G において隣接する。
- $v_1, v_2, \dots, v_k$ の色は相異なる。

Color-Coding による 【問題 1】 の解法は、おおよそ次のようなものです。

- 以下を **十分な回数**（後述）反復する。
 - まず各頂点に 1 以上 k 以下の色を **ランダムに割り当てる**。
 - 【問題 2】 あるいは 【問題 3】 を解く。条件を満たす頂点列が見つかったら、それは 【問題 1】 の解でもあるので、これを出力し、終了する。
- 十分な回数反復しても解が見つからなかったら、解なしとして終了する。

この解法の過程で 【問題 2】 あるいは 【問題 3】 を解く必要があります。これは Color-Coding という手法とは直接は関係がないので、最小限の解説にとどめます。

▶ 【問題 2】 の解法

▶ 【問題 3】 の解法

なお、以下の解説を理解する上では、【問題 3】 の解法が理解できなくとも、【問題 2】 の解法が理解できていれば十分です。

3.3. 試行回数と成功確率

各頂点に 1 以上 k 以下の色をランダムに割り当てたとき、次が成り立つことが分かります。

- 【問題 1】 が解を持つとき、確率 $\frac{1}{k^k}$ 以上で 【問題 2】 に解が存在する。
- 【問題 1】 が解を持つとき、確率 $\frac{k!}{k^k}$ 以上で 【問題 3】 に解が存在する。

これは、【問題 1】の解をひとつ固定した上で、そのひとつの解が【問題 2】や【問題 3】で検出できる確率を考えると分かります。（なお、確率いくつ以上という下からの評価になっているのは、【問題 1】が複数の解を持つときのことを考慮しているからです。また【問題 2】で $k \geq 2$ の場合には、頂点の並び順が逆順である場合を考えることで、確率を $\frac{2}{k^k}$ 以上と見積もることもできます。）

このことから 3.2 節のアルゴリズムの試行回数と成功確率の関係を調べることができます。成功確率 p の試行を n 回反復するとき、1 度も成功しないという失敗確率は $(1 - p)^n$ です。例えば $k = 5$ として試行回数と失敗確率の関係をプログラムで調べると、次の表のようになります。

$k = 5$ のとき試行回数と失敗確率の関係					
試行回数	50	100	150	200	250
失敗確率	0.141	0.020	0.0028	0.00039	0.000056
失敗確率	0.1	0.01	0.001	0.0001	0.00001
試行回数	59	118	177	236	295

なお、このような見積もりでは**自然対数** e に関する知識も役に立ちます。

▶ 自然対数の底 e とは

p が小さいとき、 n/p 回程度の試行で失敗確率 e^{-n} となります。このことから、目標の失敗確率に対してどのように試行回数 n を定めればよいかが分かります。

x	1	2	3	4	5
e^{-x}	0.368	0.135	0.0498	0.0183	0.00673
e^{-x}	0.1	0.01	0.001	0.0001	0.00001
x	2.30	4.61	6.91	9.21	11.5

また、試行回数（実行時間）を 2 倍、3 倍と増やすにつれて、失敗確率は 2 乗、3 乗という指数の関係で減少することにも注意してください。ある実行時間での失敗確率が $\frac{1}{10}$ であるとき、その実行時間を 10 倍、100 倍とすると失敗確率が $\frac{1}{10^{10}}$ や $\frac{1}{10^{100}}$ であるようなアルゴリズムになります。

▶ 参考：乱択アルゴリズムの失敗確率と競技プログラミング

【問題 3】 を利用する解法では、試行 1 回あたりの成功確率は $\frac{k!}{k^k}$ で、また 1 回の試行あたりの計算量が $O(2^k(N + M))$ 時間でした。このことから、次の定理が得られます。

【定理 4】

$\varepsilon > 0$ を正実数とするとき、Color-Coding によって 【問題 1】 を次の成功確率と時間計算量で解くことができる。

- 成功確率 $(1 - \varepsilon)$ 以上。
- 時間計算量 $O\left(\frac{(2k)^k}{k!}(N + M) \log \varepsilon^{-1}\right)$ 。

3.4. k 頂点のパスに関する最適化

上に挙げた解法は、ほとんどそのまま次のような最適化（最大化や最小化）の問題の解法になります。

【問題 5】

G を N 頂点 M 辺の単純無向グラフとします。さらに、各辺には重みが定まっています。 k を正整数とします。

G に k 頂点のパス（長さ $k - 1$ のパス）であって、通る辺の重みの総和が最大であるものを求めてください（そのようなパスが存在しなければそのことを報告してください）。

頂点にランダムな色を割り当てた上で 【問題 2】 や 【問題 3】 のような最適化問題を解くことを十分な回数反復し、それらの解の最大値を出力する、というようにすればよいです。成

功確率の評価についても 【問題 1】 と同様です。この方法で最適解が見つかるのは、最適解を与えるパスが一度以上探索対象になるときだからです。

3.5. 発展的な話題

本講座では、 k 頂点のパスの検出あるいは最適化という問題を通して、Color-Coding という手法を解説しました。ここで、 k 頂点のパスの検出に関する他の話題について少し触れておきます。

【問題 1】 や 【問題 5】 は、一般には **NP 困難**であることが知られています。本講座のアルゴリズムは、これらの問題には、 k を定数と見なせば N, M については多項式時間の計算量で解けるというもので、 k をパラメータとする **FPT アルゴリズム** (Fixed Parameter Tractable Algorithm) として有名です。

【定理 4】 において k の関係する部分 $\frac{(2k)^k}{k!}$ を、 $\frac{(2k)^k}{k!} = O^*((2e)^k)$ と評価することもできます。この指数を改良するという研究も行われています。

例えば、本講座の手法をほとんどそのまま拡張して、頂点に割り当てる色の種類数を k よりも少し大きくとることで、【問題 1】 はより高速な計算量 $O^*(4.32^k \cdot (N + M) \log \varepsilon^{-1})$ 時間で解くことができることが分かります (文献 4)。

なお、Color-Coding とは全く異なる方向の乱択アルゴリズムにより計算量を $O^*(2^k(N + M) \log \varepsilon^{-1})$ に改良できることも知られています (文献 5)。

また、本講座の解法はいくら反復を増やしても、解の正しさを確率的にしか保証できない点が気になった方もいると思います。そこで、乱択をやめて k 個の色の割り当てを戦略的に反復することで、すべてのパスが 【問題 2】 や 【問題 3】 の解として発見できるようにするという方法も考えられます。このような方法は**脱乱択化** (derandomization) と呼ばれます。

脱乱択化によって解の正しさを保証するための反復回数は、乱択する場合の解法より大きくなりますが、 N に対する依存度は $O(\log N)$ 程度に抑えられることが知られています (文献 1)。

4. 関連問題

本講座では、 k 頂点のパスの検出 (最適化) という問題に限定して、Color-Coding という手法を解説しました。他にも

- k 個のもの $a_1, a_2, \dots, a_k$ を選ぶというタイプの最適化問題.
- ただし各 a_i が相異なる, あるいは各 a_i の種類が異なるという制約がある.

というような最適化問題について, Color-Coding が有効に働くことがあります.

1. <https://yukicoder.me/problems/no/408>
2. https://atcoder.jp/contests/abc429/tasks/abc429_e
3. <https://codeforces.com/contest/2003/problem/F>
4. <https://ac.nowcoder.com/acm/contest/33195/K>
5. https://atcoder.jp/contests/stpc2025_1/tasks/stpc2025_1_l
6. https://atcoder.jp/contests/abc447/tasks/abc447_g

▶ 問題について

5. まとめ

本講座では, k 頂点のパスの検出という問題を通して Color-Coding について解説しました.

Color-Coding は, 各要素に色を割り当てることで, 「相異なる k 種類の要素を使う」というようなタイプの制約がある最適化問題を, はじめからものの種類が k 種類である場合に帰着できるという手法です. ある程度の確率で正しく解ける解法を何度も反復すれば正しい解が求まるという乱択アルゴリズム特有の解法も面白いところだと思います.

また, Color-Coding は学術的にはグラフから k 頂点のパスやサイクル, その他の種類の部分グラフを検出するアルゴリズムとして提案されたアルゴリズムですが, その用途は部分グラフ検出だけにとどまらないことが関連問題からも分かると思います.

6. 参考文献

1. Noga Alon, Raphael Yuster, Uri Zwick. Color-Coding. Journal of the ACM. Vol. 42. No. 4. pp. 844–856. 1995. DOI: 10.1145/210332.210337. <https://doi.org/10.1145/210332.210337>.
2. Wikipedia. Color-Coding. <https://en.wikipedia.org/wiki/Color-Coding>. (閲覧日: 2026-03-31).

3. Wikipedia. Longest path problem.
https://en.wikipedia.org/wiki/Longest_path_problem. (閲覧日: 2026-03-31).
4. Falk Hüffner, Sebastian Wernicke, Thomas Zichner. Algorithm Engineering for Color-Coding with Applications to Signaling Pathway Detection. Algorithmica. Vol. 52. No. 2. pp. 114–132. 2008. DOI: 10.1007/s00453-007-9008-7. <https://doi.org/10.1007/s00453-007-9008-7>.
5. Ryan Williams. Finding paths of length k in $O^*(2^k)$ time. Information Processing Letters. Vol. 109. No. 6. pp. 315–318. 2009. DOI: 10.1016/j.ipl.2008.11.004. <https://doi.org/10.1016/j.ipl.2008.11.004>.
6. Yuichi Yosida. 乱択アルゴリズム紹介(Color-Coding). Preferred Networks Tech Blog. <https://tech.preferred.jp/ja/blog/color-Coding/>. (閲覧日: 2026-03-31).
7. 清水 伸高. color-codingと脱乱択化. Le Algorithm. <https://lealgorithm.blogspot.com/2018/11/color-coding.html>. (閲覧日: 2026-03-31).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)